# PRODUCT_SPEC

Status: Draft v1. Owner: you. Language: simple English.

## 1. What Obsidian is

Obsidian is a human-gated **enhancer** for GitHub repositories. A logged-in user selects one of their own public repositories. Obsidian reads it (without running any of its code), reports bugs, security issues and enhancement opportunities, proposes an enhancement plan, applies only what the user approves, shows an exact diff, and pushes to a separate branch and opens a pull request only after the user gives permission.

Obsidian is an enhancer, not a bug fixer. Bugs and errors are only reported.

## 2. Users

Any GitHub user (open signup), from any device through a web link. Each user can use only their **own personal public repositories**.

## 3. Locked decisions

| ID | Decision |
|---|---|
| D1 | Open signup with GitHub OAuth. Anyone can log in. |
| D2 | Only the user's own personal repos. Not other people's repos, not organization repos. |
| D3 | Public repos only. Private repos are rejected (including the user's own, default). |
| D4 | **Repository code is never executed.** No install, build, test, lint, scripts or git hooks. Static analysis only. |
| D5 | No sandbox, no baseline, no re-test by running code. |

| ID | Decision |
|---|---|
| D6 | Work is saved on a separate branch obsidian/enhance-REPO in the user's own repo, never on main. The server keeps files only temporarily while working. Write access is requested at login. |
| D7 | Never push to main. Never force-push. Never auto-merge. The user merges. |
| D8 | Findings are reports only. Obsidian does not fix bugs by itself. |
| D9 | AI only suggests. Login, approval, state and security checks are normal code. |
| D10 | Obsidian's own tech stack (how Obsidian is built): React + Vite (JS), Node + Express (JS, no TypeScript), MongoDB, BullMQ + Redis, Python AI service. This does not limit user repos. Obsidian analyzes and enhances repos in any language or stack. Syntax checks for a language without a parser are marked unchecked. |
| D11 | Folders: frontend, backend, aiServices (plus docs). |
| D12 | Work is stored on the GitHub work branch. Server disk is temporary only. No cloud storage. Deployment on Render later. |
| D13 | CI/CD feature is out of scope for now. |

## 4. Workflow (F1 to F14)

1. **F1 Login** with GitHub OAuth.
2. **F2 Account check**: user must be logged in and not banned. Otherwise deny and return to login.
3. **F3 Repo select**: user picks one repo. Server checks owner equals logged-in user, repo is public, repo is personal.

4. **F4 Temporary snapshot and GitHub work branch**: clone at an exact commit (SHA) with hooks disabled. Create the work branch obsidian/enhance-REPO on GitHub (same commit as the pinned SHA, suffix -2, -3 if the name exists). The server copy is temporary.
5. **F5 Analyze**: detect languages, frameworks and package managers from file content, not folder names. Deterministic. AI only helps explain.
6. **F6 Findings**: bugs, security issues, enhancement opportunities with file and line evidence. Report only.
7. **F7 Plan**: user chooses what to enhance (files, risk, UI, animation, performance). Each operation lists files, risk and expected change.
8. **F8 Approval**: per plan. High-risk operations need extra confirmation. Bound to user, repo, SHA and operations.
9. **F9 Execute**: backend applies only approved file edits to the snapshot.
10. **F10 Version 2**: approved edits are committed to the work branch, never main. Lineage records the source SHA, approved operations and commit SHAs.
11. **F11 Static checks**: syntax parse, diff-vs-manifest match, no unapproved file changed. Code is not run.
12. **F12 Review**: exact diff, change manifest, check results, warnings.
13. **F13 Push permission**: user explicitly allows opening the pull request.
14. **F14 Push and PR**: re-verify repo, branch and SHA, then open a PR from the work branch with base main.

Note: the earlier F6 baseline hard gate is removed because code is no longer executed.

## 5. Functional requirements and acceptance criteria

| ID | Requirement | Acceptance criteria |
|---|---|---|
| FR-1 | GitHub OAuth login | Session is an httpOnly cookie. GitHub token never reaches browser JavaScript. |
| FR-2 | Banned or logged-out users blocked | Protected APIs return 401 or 403, tested with direct API calls. |
| FR-3 | Own public personal repo only | Another user's repo, private repo and org repo are rejected by the server even if the UI sends them. |

| ID | Requirement | Acceptance criteria |
|---|---|---|
| FR-4 | Snapshot at exact SHA | SHA stored on the workflow. The only GitHub write before approval is creating the work branch. No write ever targets main. |
| FR-5 | Content-based detection | Same result when folder names change (frontend/backend vs apps/packages). |
| FR-6 | Findings with evidence | Every finding has file path and line or evidence. Unproven items are labelled uncertain. No finding changes code. |
| FR-7 | Plan limited to snapshot files | An operation touching a file outside the snapshot or outside the plan is rejected. |
| FR-8 | Approval binding | SHA change makes approval stale and rejected. Approval for one plan cannot be reused for another. Double click does not execute twice. |
| FR-9 | Execute only approved ops | Path traversal and out-of-scope edits are rejected. Tested with malicious paths. |
| FR-10 | V2 separate from original | Lineage record links original SHA, approved operations and V2. |
| FR-11 | Static checks | Files in diff equal files in manifest. Syntax errors in changed files are flagged. |
| FR-12 | Honest review page | Page states: Obsidian did not run this code, test it yourself before merging. |
| FR-13 | Push permission | No permission means no PR is opened. Changed SHA at PR time marks the workflow stale and rejects it. Commits go only to the work branch. |
| FR-14 | Branch safety | No code path pushes to main, force-pushes or merges. Branch name collision adds suffix -2, -3. |

## 6. Security requirements (summary, details in THREAT_MODEL)

Never execute repo code. Hooks disabled on clone. Symlinks not followed. Size and file-count limits. Token only in backend, encrypted. AI has no secrets from GitHub. AI output is untrusted and validated. Per-user data isolation. Rate limits and per-user AI cost limit. Ban option. User data delete.

## 7. Operational requirements

Workflow state lives in MongoDB, Redis only holds the job queue. State changes use conditional updates. Retries are safe. Cancel, timeout and crash recovery are defined. Temporary files are cleaned on failure and on startup.

## 8. Out of scope

Running builds or tests, sandbox, private repos, other people's repos, organization repos, auto bug fixing, auto-merge, CI/CD and deployment features, cloud storage.

## 9. Open decisions

| ID | Question | Current default |
|---|---|---|
| O1 | AI provider, model, monthly budget | Decided later. Keys go in .env. |
| O2 | Exact GitHub OAuth scopes. Write access is needed at login now. A classic OAuth write scope may cover all the user's public repos; a GitHub App can limit it to chosen repos. | Verify in official GitHub docs before building. |
| O3 | Render plan, MongoDB and Redis hosting, disk persistence | Decide at deployment. Not verified. |
| O4 | Static parser libraries per language | Decide in architecture phase. |
| O5 | Per-user limits (daily workflows, repo size) | Numbers to be set. |
| O6 | Own private repo | Rejected. |

## 10. Risks

Static analysis cannot prove code works, so Obsidian never claims tested or secure. Prompt injection from repo text. Open signup abuse and AI cost. Solo developer scope, so build in phases with approval after each phase.